

SLMP and Binary Search and some ideas about algorithm performance

Tanvir Ahmed, PhD

Department of Computer Science,

University of Central Florida

Motivation

- One important focus of this course is to write an efficient code, understanding and analyzing the run-time and performance of the codes, etc.
- As part of it, in this note we will try to understand how a simple problem can be solved in many different ways, the concept of performance, and how we can improve the solutions to do it with less number of steps.
- As part of it, we will do:
  - Linear Search
  - Binary Search
  - Sorted list matching problem
    - Brute force solution
    - Using binary search in the process
    - More efficient one without using binary search

Linear/sequential search

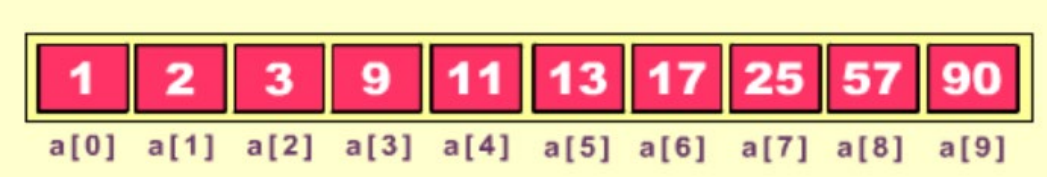
- Consider the following sorted array and you would like to search for 57.
  - int array[10] = {1, 2, 3, 9, 11, 12, 17, 25, 57, 90}.
- A naïve approach would be sequentially lookup each item from the beginning of the array until you find the item or you reach to the end of the array.
  - So, in this case, you will have to access 9 items from your array to find 57.
  - If the item does not exist, you might need to access all the array elements to make the decision
- This approach is linear or sequential search and in worst case you might need to access all the array element to make a decision whether you will find the item or not.
- The linear search code will look like this:

```
int LinearSearch(int array[], int len, int value)
{
        int i;
        for (i=0; i<len; i++)
        {
                if (array[i] == value)
                        return 1;
        }
        return 0;
}
```

- Clearly, if your array is unsorted, the above solution is optimal as you don't know what part of the array has the item unless you look one after another.
  - If you have 1000 items, in worst case, you will have to access 100 items
  - If you have 2000 items, in worst case, you will have to access 2000 items
  - If you have n items, in worst case, you will have to access n items
  - So, the total amount of steps increases linearly. We call it Order of n => O(n)
- Can you do better if we have a sorted array?

Binary Search

- If you know that the array is sorted, we can guess better what part of the array the data should be located
- For example see the following array:

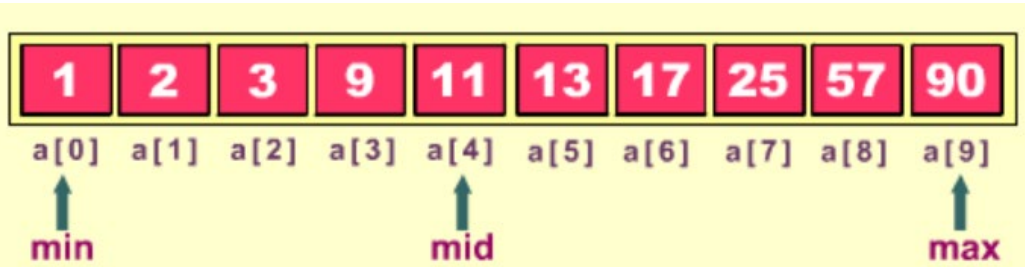


- If you want to search for 57, we can directly start our search in the upper half of the array
- That upper half of the array also can be treated as another array and we can even look upper half of that new array
- and so on….
- We divide our search space like this until we find the item or we are sure that our item does not exist
- So, what is the mid point of the above array?
  - (left most index + right most index)/2 = (0+9)/2 = 9/2 = 4
- So, we need two numbers, the ***low index and high index*** and calculate:
  - mid index = (low index + high index)/2
- What would be your mid point if your low=4 and high = 9?
  - (4+9)/2 = 6
- This approach of searching is very intuitive when searching in a sorted list.
  - Let’s see an step by step example in the next slide

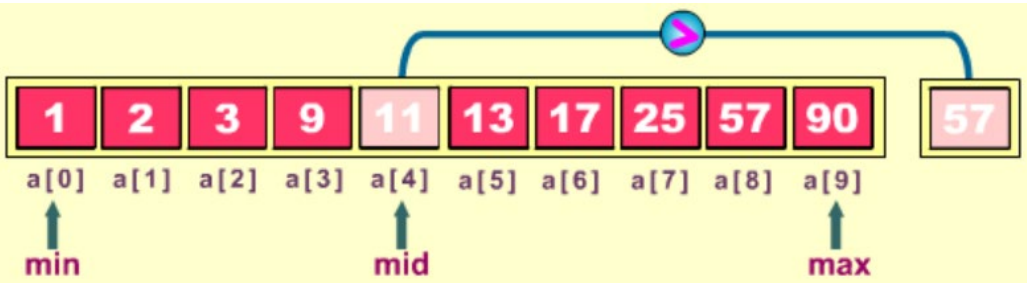
Binary Search Simulation



Suppose the data that is to be searched is 57

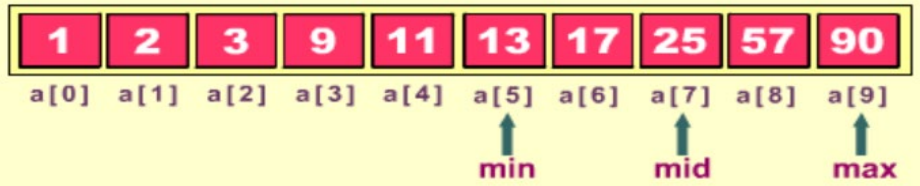


mid = (min + max) / 2
mid = (0 + 9) / 2
mid = 4



a[mid] is compared with 57

As 11 is less than 57, lower bound min is pointed to a[5]



mid = (min + max) / 2
mid = (5 + 9) / 2
mid = 7



a[mid] is compared with 57

As 25 is less than 57, lower bound min is pointed to a[8]



mid = (min + max) / 2
mid = (8 + 9) / 2
mid = 8

a[mid] is compared with 57

Search is successful as 57 is found

How about if the item search is 58?

Binary search. Search for item = 20 in the bellow array:

0	1	2	3	4	5	6
-15	18	20	25	30	35	112

- Lets work with the array index:

  l = 0 and h = 6. So, mid = (l+h)/2 = 3

- What is in Array[mid]?
  - It is 25.
  - It is > 20.
  - So, our item is actually before it.
  - So, our new h = mid-1 = 2.
  - New mid = (0+2)/2 = 1
  - Now, Array[mid] = 18 < 20
  - So, our item is actually after mid.
  - So, l = mid + 1 = 2
  - Array[mid] = 20 == item. So, return 1 (success)
- Let's try searching for 32
- **How would you know that the item is not available?**
  - **If low>high**

Binary search

- So, during the iteration, we update our low or high based on the condition
- Because we want to check the data in that part of the array and we want new mid

```
int binarySearch(int list[], int item, int len)
{
    int l = 0, h = len - 1;
    int mid;
    while (l <= h)
    {
        mid = (l + h) / 2;
        // Check if item is present at mid
        if (list[mid] == item)
            return mid;
         // If item greater, ignore left half
        if (list[mid] < item)
            l = mid + 1;

        // If item is smaller, ignore right half
        else
            h = mid - 1;
    }
    // if we reach here, then element was
    // not present
    return -1;
}
```

```
#include <Stdio.h>

int binarySearch(int list[], int item, int len)
{
    int l = 0, h = len - 1;
    int mid;
    while (l <= h)
    {
        mid = (l + h) / 2;
        // Check if item is present at mid
        if (list[mid] == item)
            return mid;
         // If item greater, ignore left half
        if (list[mid] < item)
            l = mid + 1;

        // If item is smaller, ignore right half
        else
            h = mid - 1;
    }
    // if we reach here, then element was
    // not present
    return -1;
}


int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int searchitem = 10;

    int result = binarySearch(arr, searchitem, 4);
    (result == -1) ? printf("Element is not present"
                            " in array")
                   : printf("Element is present at "
                            "index %d",
                            result);
    return 0;
}
```

Analyzing the number of steps by Binary Search

- Our search space starts at n,
- After 1 step search space is no bigger than n/2
- After 2 steps, the search space is no bigger than n/4
- After 3 steps, the search space is no bigger than n/8
- So, after k steps, the search space is no bigger than $\boldsymbol{n/2^k}$
- The algorithm will stop after our search space is size 1 (once low exceeds high, this loop will not run again.
  - Let's solve the following equation for k, the number of steps this algorithm takes:
    - $\boldsymbol{n/2^k} = 1$
    - $n = \boldsymbol{2^k}$
    - $K = \log_2 n$

- Thus, binary search takes no more than roughly **$\log_2 n$ steps. $O(\log_2 n)$**
- So, if we're searching in 2 million items, we will only make 20 comparisons, at most!
- HUUUGGEEE saving!

Binary search recursive code (we will do it after learning some recursions)

```
int binSearch(int *values, int low, int high, int searchval)
{
        int mid;
        if (low <= high)
        {
                mid = (low+high)/2;
                if (searchval < values[mid])
                        return binSearch(values, low, mid-1, searchval);
                else if (searchval > values[mid])
                        return binSearch(values, mid+1, high, searchval);
                elsereturn 1;
        }
        return 0;
}
```

Sorted List Matching Problem (SLMP)

- Given two sorted lists of distinct numbers, output the numbers common to both lists.
- How would you attack the problem?
  - For each number on list 1, do the following:
    - a) Search for the current number in list 2.
    - b) If the number is found, output it.

- If a list is unsorted, steps a and b (which is simply a linear search) might take n steps (n = number of elements in list 2)

SLMP

- If you don't use the information that the list is sorted, we can do a brute force solution:

```
void printMatches(int list1[], int list2[], int len1, int len2)
{
        int i,j;
        for (i=0; I < len1; i++)
        {
                for (j=0; j<len2; j++) //linear search
                {
                        if (list1[i] == list2[j])
                        {
                                printf("%d", list1[i]);
                                break;
                        }
                }
        }
}
```

- How many steps it might take in total, if the size of list1 = n and size of list2 = n?
- n^2

SLMP

- But we know both lists are already sorted.
- Thus, we can use binary search in step a.
- Assuming both list are of same sizes, the binary search takes about log n steps.
- We have to repeat n times. So, total around *n log n.* Much better than n^2.

Going back to SLMP

Enhance your code to find common items in two arrays (n * log n)
So, just using binary search in our last SLMP code can result in n * log n as binary search works for log n and we want to use binary search n times.
#O(n * log n)

#Can you even improve it further???:

O(n) SLMP!

Consider how you'd probably do this task in real life...

List #1	List #2
Adams	Boston
Bell	Davis
Davis	Duncan
Harding	Francis
Jenkins	Gamble
Lincoln	Harding
Simpson	Mason
Zoeller	Simpson

You'd read that Adams and Boston are the first names on the list. Immediately you'd know that Adams wasn't a match, and neither would any name on the list #1 alphabetically before Boston. So, you'd read Bell and go on to Davis. At this point you'd deduce that Boston wasn't on the list either, so you'd read the next name on list #2 – voila!!! A match! You'd output this name and simply repeat the same idea. In particular, what we see here is that you ONLY go forward on your list of names. And for every "step" so to speak, you will read a new name off one of the two lists. Here is a more formalized version of the algorithm:

Reference: More hints in page 3 and 4 of this link:
http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/lec/IntroAlgorithms-02.pdf

SLMP O(n)

1) Start two “markers”, one for each list, at the beginning of both lists.
2) Repeat the following steps until one marker has reached the end of its list.

a) Compare the two items that the markers are pointing at.
b) If they are equal, output the number and advance BOTH markers one spot.
If they are NOT equal, simply advance the marker pointing to the number that comes earlier one spot.

This will improve the run time and will result in 2n steps.
More hints in page 3 and 4 of this link:
http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/lec/IntroAlgorithms-02.pdf

Here is the O(n) code to the SLMP problem!

```
void slmp_linear(int list1[], int n, int list2[], int m) {

   int i = 0, j = 0;

   // Go while we still have numbers in both lists.
   while (i < n && j < m) {

      // Safe to advance list 1 pointer.
      if (list1[i] < list2[j]) i++;

      // Safe to advance list 2 pointer.
      else if (list2[j] < list1[i]) j++;

      // Match!
      else {
         printf("%d ", list1[i]);
         i++;
         j++;
      }
   }
}
```

At each step in the while loop, either i or j is increasing, that results in one scan to each of the arrays in total.
So, total steps would be (n+m) which is O (n+m).

If both array size was n, then it would be 2n => O(n) linear time. (removing any constant factors and lower order terms will give you big – O O(n))

We will do more algorithm analysis later!